RUBISCAPE

Technology & Analytics Division

PROJECT COMPLETION REPORT

Dataset Versioning and Comparison Tool

DataVault V2 Platform

Project ID	RS-DE-04
Project Title	Dataset Versioning and Comparison Tool
Platform Name	DataVault V2
Division	Data Engineering
Document Type	Project Completion Report
Status	COMPLETED — Ready for Deployment
Prepared For	Rubiscape Leadership & Technical Review
Version	v2.0 — Final

1. Executive Summary

This report documents the successful design, development, and delivery of DataVault V2 — a comprehensive, full-stack data engineering platform built under project reference RS-DE-04. The platform addresses a recurring operational challenge within data teams: the absence of structured, auditable version control for datasets used in machine learning pipelines and analytics workflows.

DataVault V2 delivers nine tightly integrated capability domains: dataset versioning with branch and merge support, statistical drift detection between dataset versions, automated multi-model ML evaluation, data quality scoring, live prediction serving, an AI-powered copilot, a best-version radar dashboard, team collaboration controls, and a full administrative panel. The platform is production-ready, deployed on a FastAPI backend with a zero-dependency React frontend, and is accessible immediately following a single-command startup.

The AI Copilot feature — a key differentiator — operates on a strict privacy-first architecture: it transmits only pre-computed statistical aggregates to external language model APIs, ensuring no raw customer or dataset records ever leave the internal environment. This design is aligned with enterprise data governance requirements.

2. Project Overview

Attribute	Detail
Project ID	RS-DE-04
Project Title	Dataset Versioning and Comparison Tool
Platform Name	DataVault V2
Division	Data Engineering
Project Category	Internal Tool — ML & Data Infrastructure
Technology Stack	Python 3.13 / FastAPI / React (CDN) / SQLite / Parquet
External Dependencies	None at runtime (pip install only)
Deployment Mode	Local / On-premise (port 8000)
Default Credentials	admin / Admin@123 (change on first login)
Delivery Status	COMPLETE — All modules functional

3. Business Context & Problem Statement

Data engineering teams working on ML projects routinely iterate over datasets — cleaning, enriching, filtering, and augmenting — yet most organisations lack a disciplined mechanism to track what changed between iterations, evaluate the downstream ML impact of each change, or confidently identify which dataset version produced the best model. This creates three compounding risks:

- Reproducibility gaps: teams cannot reliably recreate the dataset state that produced a specific model or result.
- Blind upgrades: newer dataset versions are promoted to production without statistical validation of distributional drift or quality regression.
- Tribal knowledge dependency: dataset history exists in individual engineers' memories or ad-hoc file naming conventions rather than a shared, searchable system.

DataVault V2 eliminates these risks by providing Git-like version control semantics for datasets, paired with automated quality and ML performance evaluation so that every version decision is grounded in quantitative evidence rather than intuition.

4. System Architecture

The platform follows a clean two-tier architecture: a Python-based backend exposing a REST API, and a browser-rendered React frontend requiring no build toolchain.

4.1 Backend — FastAPI Application

- Entry point: main.py — initialises the database, configures CORS, registers all API routes.
- Core modules: auth.py (JWT, 48-hour tokens), database.py (SQLite + WAL mode), versioning.py, diff_engine.py, ml_engine.py, quality.py, predictor.py, copilot.py.
- API layer: routes.py — all endpoints under /api prefix, fully documented at /docs (Swagger UI).
- Storage: CSV uploads are converted to Apache Parquet for compressed, columnar on-disk storage under data/storage/.

4.2 Frontend — React (CDN, Zero Build)

- Single-file React application loaded entirely from CDN — no npm, no webpack, no Node.js required at runtime.
- Tabs: Overview, Versions, Compare, Evaluate, Best Radar, Predict, Copilot, Team.
- Communicates with the backend via a shared api() helper with JWT Bearer token authentication.

4.3 Data Flow

- Upload: CSV → pandas read → Parquet write → schema and statistics extracted and stored in SQLite.
- Quality: computed synchronously at upload time; stored in quality_scores table.
- ML Evaluation: runs asynchronously in a background thread; frontend polls status every 2 seconds.
- Predictions: model trained in-memory on demand; results cached for the session.
- Copilot: pre-computed statistics assembled from SQLite; system prompt built and sent to LLM API — no raw data leaves the server.

5. Delivered Features

5.1 Dataset Versioning

Every CSV uploaded to a project is saved as a versioned, immutable Parquet snapshot. The system supp orts named branches (main, experimental, hotfix, etc.) with full lineage tracking. Versions carry structure d metadata: commit message, author, branch, parent version, row count, column count, schema, and per -column statistics. Merge operations support three configurable strategies — latest, best_quality, and lar gest — enabling teams to promote dataset branches in a controlled, auditable manner.

5.2 Drift Detection & Statistical Comparison

Any two versions of a dataset within a project can be compared side by side. The diff engine produces th ree classes of analysis:

- Schema diff — columns added, removed, or type-changed between versions.
- Row statistics — absolute and percentage change in row count; duplicate row counts per version.
- Statistical drift — KS test and PSI (Population Stability Index) for numeric columns; Chi-squared te st for categorical columns. Each column is assigned a drift level (NONE, LOW, MEDIUM, HIGH) an d the overall comparison is assigned a severity rating (NONE through CRITICAL).

Results are persisted in the database and cached so that repeated comparisons are instantaneous.

5.3 ML Evaluation Engine (10 Models + Ensemble)

Any dataset version can be submitted for full ML evaluation against a user-specified target column. The e ngine automatically determines task type (classification or regression) based on the target column's cardi nality and data type, then trains and evaluates ten models:

Model	Classification Metric	Regression Metric
Random Forest	Accuracy, F1, AUC-ROC, CV	R^2, RMSE, CV
Gradient Boosting	Accuracy, F1, AUC-ROC, CV	R^2, RMSE, CV
XGBoost	Accuracy, F1, AUC-ROC, CV	R^2, RMSE, CV
Logistic / Linear Reg.	Accuracy, F1, AUC-ROC, CV	R^2, RMSE, CV
SVM / SVR	Accuracy, F1, AUC-ROC, CV	R^2, RMSE, CV
K-Nearest Neighbours	Accuracy, F1, AUC-ROC, CV	R^2, RMSE, CV
Naive Bayes	Accuracy, F1, AUC-ROC, CV	— (placeholder)
Decision Tree	Accuracy, F1, AUC-ROC, CV	R^2, RMSE, CV
Extra Trees	Accuracy, F1, AUC-ROC, CV	R^2, RMSE, CV
MLP Neural Network	Accuracy, F1, AUC-ROC, CV	R^2, RMSE, CV

An ensemble result is computed post-evaluation using both simple average and CV-score-weighted avera ge across all successfully trained models. Feature importances are aggregated across models and normali sed. Results are stored and exposed via a polling API so the UI remains non-blocking during training.

5.4 Best Version Radar Dashboard

Once one or more evaluations have been run across a project's versions, the Best Radar tab renders six coordinated visualisations: a Champion Card (trophy + best model + top scores), a Performance Timeline (all model scores per version with an ensemble trend line), a Radar/Spider chart (top-5 versions on six axes), a Model Win Counter, an Ensemble vs. Best Single bar chart, and a Quality Heatmap across all four quality dimensions. These charts give stakeholders a concise, evidence-based answer to the question: which version of this dataset should we use?

5.5 Data Quality Scoring

Every uploaded version is automatically assigned a quality score across four dimensions, each computed from the raw data without requiring any configuration:

- Completeness (35% weight) — proportion of non-null cells across the entire dataset.
- Uniqueness (25% weight) — inverse of the duplicate row rate.
- Consistency (25% weight) — penalises columns with IQR-based outlier rates exceeding 30%.
- Validity (15% weight) — flags impossible values in domain-specific columns (age, year, percentage, ratio).

The weighted overall score maps to a letter grade (A through F) and is stored in the database, surfaced in version lists, and used by the merge strategy selector when best_quality is chosen.

5.6 Live Prediction Service

Users can train a prediction model on any versioned dataset with one click and immediately submit single-row predictions through a dynamically generated form (field types inferred from schema) or upload a CSV for batch prediction. Batch results are downloadable as a labelled CSV. The predictor includes a verdict generator that produces human-readable outputs for common target column semantics (churn, survival, fraud/default).

5.7 AI Copilot — Privacy-First Design

The Copilot tab provides a conversational AI assistant that answers questions about dataset health, model performance, feature importance, and drift — grounded in the project's actual computed metrics. The architecture is built around a strict data minimisation principle:

- Only pre-computed aggregates from the database are sent to the LLM API: quality scores, column-level statistics (mean, std, min, max, median), model leaderboard scores, feature importances, and drift highlights.
- Raw CSV rows, Parquet file contents, and individual data records are never transmitted to any external endpoint.

Five LLM providers are supported, each with correct model names and API schemas:

Provider	Model	Key Prefix	Notes
xAI Grok (default)	grok-beta	xai-...	Fallback: grok-2-mini, grok-2-121

			2
Anthropic Claude	claude-3-haiku-20240307	sk-ant-...	Own schema; system prompt separate
OpenAI GPT	gpt-4o-mini	sk-...	Fallback: gpt-3.5-turbo
Google Gemini	gemini-2.0-flash	AIza...	Fallback: 3 models; 429=rate limit
DeepSeek	deepseek-chat	sk-...	402=insufficient balance

The Copilot UI includes a transparency control (AI Context button) that displays the exact system prompt sent to the LLM, confirming no raw data is present. Suggested questions are generated dynamically based on project state (whether evaluations and diffs have been run). Chat history persists across sessions and can be cleared on demand.

5.8 Team Collaboration

Projects support multi-user access via an invitation model with three role tiers: owner (full control including deletion and role management), editor (can upload versions and trigger evaluations), and viewer (read-only access to all project data). Role enforcement is applied at the API layer on every protected endpoint.

5.9 Administration Panel

The admin panel (accessible to users with the admin role) provides a system-wide view: user count, project count, version count, active evaluations, and total storage consumed. It includes a user management interface for changing roles and a live activity log showing the last 100 actions across all users and projects.

6. API Reference Summary

All endpoints are available at http://localhost:8000/api with interactive Swagger documentation at /docs. Authentication uses Bearer JWT tokens issued at login (48-hour validity).

Method	Endpoint	Description
POST	/api/auth/register	Register new user; returns JWT token
POST	/api/auth/login	Authenticate; returns JWT token
GET	/api/projects	List all projects for current user
POST	/api/projects	Create a new project
PATCH	/api/projects/{id}	Update project metadata
GET	/api/projects/{id}/stats	Version, evaluation, and storage counts
GET	/api/projects/{id}/radar-data	All Best Radar chart data
POST	/api/versions/commit	Upload CSV; creates versioned snapshot
GET	/api/versions/{id}/preview	First 100 rows of a version
GET	/api/versions/{id}/quality	Quality scores for a version

GET	/api/versions/{id}/profile	Column-level statistics and histograms
POST	/api/diff	Compute diff between two versions
POST	/api/evaluations	Start background ML evaluation
GET	/api/evaluations/{id}/status	Poll evaluation progress
GET	/api/evaluations/{id}	Full evaluation results
POST	/api/predict/train	Train prediction model on a version
POST	/api/predict/single	Submit single-row prediction
POST	/api/predict/batch	Batch CSV prediction; returns CSV
POST	/api/copilot/chat	Send message to AI Copilot
GET	/api/copilot/context/{project_id}	View exact data sent to LLM
GET	/api/copilot/suggested-questions/{id}	Context-aware question suggestions
DELETE	/api/copilot/history/{project_id}	Clear conversation history

7. Technology Stack & Dependencies

Component	Technology / Version	Purpose
Runtime	Python 3.13.0	Core backend language
Web Framework	FastAPI 0.111.0	REST API, Swagger docs, async support
ASGI Server	Uvicorn 0.30.1	Production-grade async server
Data Processing	pandas 2.2.3	CSV ingestion, schema analysis
Numerical	NumPy 2.1.1	Statistical computations
Statistical Tests	SciPy 1.14.1	KS test, Chi-squared, PSI
ML Framework	scikit-learn 1.5.2	10 ML models, cross-validation
Gradient Boosting	XGBoost 2.1.1	High-performance tree boosting
Columnar Storage	PyArrow 18.0.0	Parquet read/write
Authentication	PyJWT 2.9.0	JWT token issuance and verification
HTTP Client	httpx 0.28.1	Async LLM API calls
File Uploads	python-multipart 0.0.9	Multipart form data (CSV uploads)
Config	python-dotenv 1.0.1	Environment variable management
Database	SQLite (built-in)	WAL mode; all metadata storage
Frontend	React 18 (CDN)	No build toolchain required

8. Security & Data Governance

8.1 Authentication & Authorisation

- All API endpoints (except /auth/register and /auth/login) require a valid JWT Bearer token.
- Tokens are signed with a configurable secret key (JWT_SECRET environment variable) using HS2 56.
- Token expiry is set to 48 hours; expired tokens are rejected with HTTP 401.
- Role-based access control is enforced at the API layer for every project operation.

8.2 AI Copilot Data Governance

- No raw dataset rows are ever transmitted to LLM provider APIs.
- The system prompt contains only pre-computed aggregates: means, standard deviations, quality percentages, model CV scores, and drift statistics.
- API keys for LLM providers are never persisted server-side; they are stored client-side only in bro wser localStorage and transmitted per-request.
- The /api/copilot/context endpoint provides a full audit log of what data the AI received for any giv en project.

8.3 Storage Security

- Dataset files are stored as Parquet files on the local filesystem under the data/storage/ directory.
- The database uses WAL (Write-Ahead Logging) mode and enforces foreign key constraints.
- Passwords are stored as SHA-256 hashes; no plaintext passwords are ever persisted.

9. Deployment & Operations

9.1 Quick Start

Windows: double-click START.bat. The batch file checks Python availability, installs dependencies, create s the storage directory, seeds the admin account, and launches the server at http://localhost:8000.

Mac / Linux:

- cd backend
- pip install -r requirements.txt
- python main.py

9.2 Configuration

Variable	Default	Purpose
JWT_SECRET	datavault-secret-key-2024	Token signing key — change in product ion
PORT	8000	Modify in START.bat (--port 8001)

10. Known Issues & Mitigation

Issue	Root Cause	Mitigation / Status
Gemini 429 Too Many Requests	Free tier limit: 15 requests/minute	User shown clear wait message; upgrade link provided
Grok 400 Bad Request	Model grok-3-mini does not exist in xAI API	Fixed: fallback chain: grok-beta > grok-2-mini > grok-2-1212
OpenAI gpt-3.5-turbo deprecated	Model being phased out by OpenAI	Fixed: primary model updated to gpt-4o-mini with fallback
XGBoost install failure	Optional native binary dependency	Documented; engine degrades gracefully to 9 models
Large CSV memory spikes	pandas read_csv loads entirely in-memory	Acceptable for datasets < 500MB; chunked read planned

11. Recommended Next Steps

The following enhancements are suggested for consideration in subsequent development phases:

- PostgreSQL backend — replace SQLite with PostgreSQL to support concurrent multi-user write workloads in production environments.
- Containerisation — package the backend as a Docker image for reproducible deployment and horizontal scaling.
- Role-based UI rendering — hide tabs and controls that are inaccessible to the current user's role (currently enforced at API level only).
- Dataset chunked ingestion — process large CSVs in chunks to reduce peak memory usage and enable uploads exceeding 500 MB.
- Automated re-evaluation triggers — optionally re-run ML evaluation when a new version is committed to a project, keeping leaderboard data current.
- Email notifications — notify team members when an evaluation completes or when a high-drift alert is raised.
- Audit trail export — expose the activity log as a downloadable CSV for compliance reporting.
- SSO / LDAP integration — enable enterprise single sign-on for organisations with centralised identity management.